

WorkflowOS — Implementation Plan

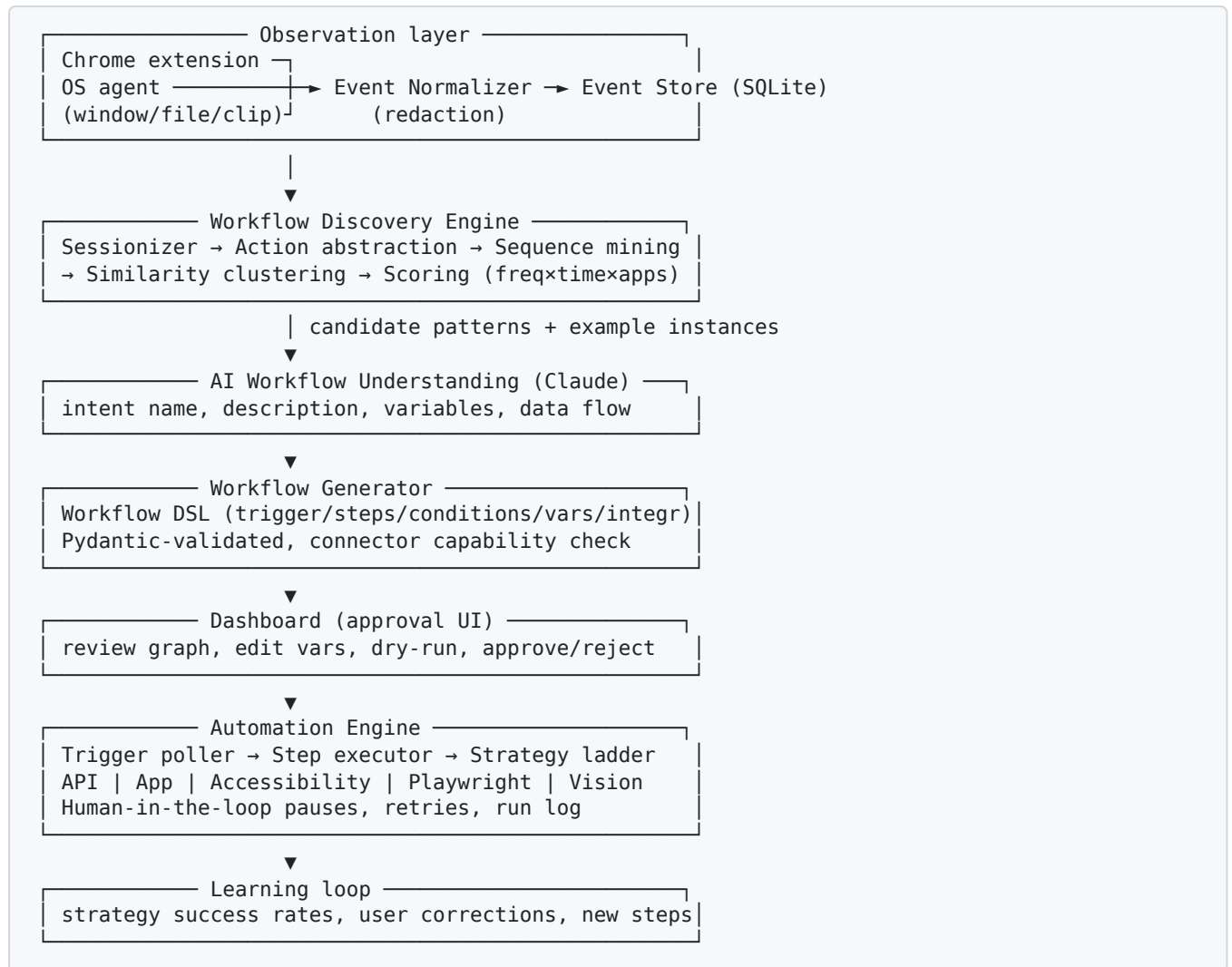
AI-powered OS-level workflow automation: **Observe → Understand → Detect Repetition → Generate Workflow → User Approval → Automate → Learn.**

This plan targets the hackathon deliverables (source ZIP + README, demo video) and is organised around the evaluation criteria: functionality, technical implementation, logic, AI & tools integration, demo quality, problem relevance, innovation.

1. Guiding decisions

Decision	Choice	Why
Core language	Python 3.11 (agent, engine, API)	Best ecosystem for OS hooks, Playwright, LLM SDKs, pattern mining
Backend API	FastAPI + WebSocket	Async, typed (Pydantic), live event stream to UI
Storage	SQLite (SQLModel)	Local-first, zero-setup, privacy friendly
Dashboard	React + Vite + TypeScript , React Flow for workflow graphs	Clear approval UX, visual workflow
Browser capture	Chrome MV3 extension	Most knowledge work (Gmail, CRM, Slack web) is in the browser; DOM-level events are far richer than pixels
OS capture	Python agent: active window (<code>pywinctl</code>), file watcher (<code>watchdog</code>), clipboard metadata	Cross-app transitions, downloads, file operations
AI	Claude API (<code>claude-opus-5-5</code> for understanding/generation, <code>claude-haiku-4-5</code> for cheap event labelling) with tool-use / JSON-schema output	Structured, validated output; intent naming; variable extraction
Execution	Strategy ladder: API → App integration → Accessibility → Playwright → Vision	Exactly the priority the problem statement asks for
Demo apps	Real Gmail API + real Slack Web API + a bundled Mini-CRM web app (REST + HTML UI)	Real, working, reproducible; CRM having both API and UI lets us show fallback live

2. Architecture



3. Repository layout

```
workflows/
  backend/
    app/
      main.py          # FastAPI app, routers, websocket
      config.py
      models/          # SQLAlchemy: Event, Session, Pattern, Workflow, Run, StepRun
      observe/
        normalizer.py  # raw → Event schema, PII redaction
        os_agent.py    # active window, file watcher, clipboard meta
      discover/
        sessionizer.py
        abstraction.py  # Event → action token ("gmail.open_email")
        mining.py       # contiguous/gapped sequence mining
        clustering.py   # edit-distance / Jaccard variant merging
        scoring.py
      understand/
        llm.py          # Claude client, prompts, tool schemas
        intent.py       # pattern → intent + variables + data flow
      generate/
        dsl.py          # Workflow Pydantic schema
        generator.py    # intent → workflow, validation, capability check
      automate/
        engine.py       # run orchestration, conditions, HITL pauses
        triggers.py     # gmail poller, schedule, manual
        strategies/
        connectors/     # gmail.py, slack.py, crm.py, filesystem.py
      learn/
        feedback.py     # strategy stats, corrections, refinement suggestions
      tests/
    extension/          # Chrome MV3: content script, background SW, popup (pause/record)
    dashboard/         # React + Vite + TS
    mini_crm/          # demo CRM: FastAPI + HTML, REST API + UI
    scripts/
      seed_demo.py     # replays recorded sessions for a quick demo
    docker-compose.yml
    README.md
```

4. Component design

4.1 Desktop Activity Agent (Observe)

Unified event schema

```
{
  "id": "uuid", "ts": "2026-09-26T10:02:11Z", "source": "browser|os",
  "app": "gmail|crm|slack|excel|finder|...", "type":
  "navigate|click|input|submit|download|file_op|window_focus|copy|paste",
  "target": {"selector": "...", "role": "button", "label": "Save", "url": "https://..."},
  "value_hash": "sha256...", "value_sample": "redacted/typed", // never raw keystrokes
  "context": {"title": "...", "entity_hints": {"email": "a@b.com"}}
}
```

- **Browser extension:** content script captures `click`, `submit`, `change` (field name + semantic label, value only for non-sensitive fields), navigation (`webNavigation`), downloads (`chrome.downloads`). App classified from URL host rules (`mail.google.com` → `gmail`).
- **OS agent:** polls foreground window every 500 ms (app switch events), `watchdog` on Downloads/Documents for file create/move, clipboard *type/length* (not contents) to detect

copy→paste data flow across apps.

- **Privacy**: runs locally; password/credit-card fields never captured; regex redaction of secrets; app allow-list; global pause toggle in extension popup and dashboard. This is a strong talking point in the demo.

4.2 Workflow Discovery Engine (Detect Repetition)

1. **Sessionize** — split the event stream into task instances on idle gap (> 3 min) or on a "task start" signal (opening a new email/ticket). Each instance is a list of events.
2. **Abstract** — map each event to an action token `app.verb.object` (e.g. `gmail.open.email`, `gmail.download.attachment`, `crm.search.customer`, `crm.update.customer`, `slack.post.message`). Rule-based for known apps, Haiku labelling for unknown pages (cached per URL pattern + selector).
3. **Mine** — frequent contiguous and gapped sub-sequences (lightweight PrefixSpan) with min support ≥ 3 instances and length ≥ 3 .
4. **Cluster variants** — merge patterns whose normalised Levenshtein distance ≤ 0.25 so "noise" steps (e.g. an extra tab switch) don't split a workflow.
5. **Score** — `score = support × avg_duration_sec × (1 + distinct_apps) × consistency`. Surface top-N as candidates.
6. **Data-flow detection** — link values across steps (same email address seen in Gmail and typed into CRM search; downloaded filename uploaded into CRM) → these become workflow **variables**.

4.3 AI Workflow Understanding (Understand)

Claude receives: the abstract pattern, 2–3 concrete (redacted) instances, and detected data links. It must respond via a tool call with a strict JSON schema:

```
{ "intent": "Process Customer Request",
  "description": "...",
  "variables": [{"name": "customer_email", "source": "gmail.sender"},
{"name": "attachment", "source": "gmail.attachment[0]"}],
  "steps": [{"action": "crm.update_customer", "inputs": {"email": "{{customer_email}}", "note": "{{email_subject}}"}},
{"conditions": [{"if": "crm.customer_not_found", "then": "pause_for_user"}],
  "confidence": 0.86 }
```

Output is validated with Pydantic; on failure we re-prompt with the validation error (max 2 retries).

4.4 Workflow Generator (Generate)

- Converts the intent into the **Workflow DSL** (stored as JSON, exportable as YAML): `trigger`, `variables`, `steps[]` (each with `connector`, `operation`, `inputs`, `outputs`, `strategies[]`), `conditions`, `on_error`.
- **Capability check**: for each step, list which strategies the connector supports and order them by the ladder; mark steps with no supported strategy as "needs user".
- Generated workflow for the reference scenario:

```

name: Process Customer Request
trigger: { connector: gmail, event: new_email, filter: "label:customer-requests" }
steps:
  - id: read_email      # identify customer
    connector: gmail; op: get_message;      out: [sender, subject, body, attachments]
  - id: get_attachment
    connector: gmail; op: download_attachment; out: [file_path]
  - id: find_customer
    connector: crm; op: find_customer; in: {email: "{{read_email.sender}}"}; out:
[customer_id]
  - id: guard
    condition: "find_customer.customer_id == null"; then: pause_for_user
  - id: update_customer
    connector: crm; op: update_customer; in: {id: ..., note: ..., file: "
{{get_attachment.file_path}}"}
  - id: notify
    connector: slack; op: post_message; in: {channel: "#support", text: "..."}

```

4.5 User Approval (Dashboard)

- **Live activity feed** (WebSocket) — shows what's being observed.
- **Discovered workflows** — card per candidate: intent, frequency ("seen 5× this week, ~4 min each → ~20 min/week saved"), confidence.
- **Workflow editor** — React Flow graph; click a step to edit inputs/variable mappings/channel; toggle strategy preference.
- **Dry run** — execute against the last observed instance with side effects disabled (connectors run in `simulate` mode) and show the planned calls.
- **Approve / Reject / Snooze**; only approved workflows are armed.
- **Runs view** — per-step status, which strategy was used, timing, screenshots for UI strategies, HITL prompts ("Customer not found — pick one or create").

4.6 Automation Engine (Automate)

- **Triggers**: Gmail poller (History API every 15 s), manual "Run now", schedule.
- **Executor**: async step runner with variable templating (Jinja2 sandbox), condition evaluation, retries with backoff, per-step timeout, persisted run state (resumable after a HITL pause).
- **Strategy ladder** per step, falling through on failure:
- **API** — Gmail API, Slack Web API, Mini-CRM REST.
- **App integration** — e.g. `openpyxl` for Excel files, local filesystem ops, app CLIs.
- **Accessibility/semantic UI** — `pywinauto` (Windows UIA) / `atomacos` (macOS AX) / AT-SPI (Linux) for desktop apps; semantic selectors (role + label) recorded by the observer.
- **Browser automation** — Playwright, replaying recorded semantic selectors (`getByRole`, `getByLabel`), with an LLM-assisted selector repair if the DOM changed.
- **Vision fallback** — screenshot + Claude computer-use style action proposal, only after user confirms (guard-railed).
- Every step logs `strategy_used`, latency, success — feeds Learning.

4.7 Learn

- Per-connector/op strategy success rate → reorder ladder (e.g. skip a flaky UI path).
- User edits during approval or HITL resolutions are stored as corrections and re-injected as few-shot examples into the understanding prompt.
- Post-run observation: if the user keeps doing a manual step right after the workflow runs (e.g. always adds a CRM tag), propose extending the workflow — "Learn" closes the loop back to Discover.

5. Demo scenario (real, working example)

Setup: test Gmail account (label `customer-requests`), Slack workspace with a bot in `#support`, Mini-CRM running locally with seeded customers.

1. **Show the output first** (per submission rules): an email arrives → within ~15 s the CRM record is updated with note + attachment and a Slack message appears in `#support`.
2. **Observe**: record the user doing the flow manually 3–4 times with different customer emails; activity feed shows structured events.
3. **Discover + Understand**: click "Analyse" → candidate "Process Customer Request" appears with graph, variables, time-saved estimate.
4. **Approve** after a dry run; arm the workflow.
5. **Automate**: send a new email → end-to-end run, steps light up, strategy badges show `API`.
6. **Fallback**: disable the CRM API in Mini-CRM settings → re-run → CRM step falls back to **Playwright** visibly driving the CRM UI.
7. **Condition/HITL**: email from an unknown customer → workflow pauses, dashboard asks the user to pick/create the customer, then resumes.
8. **Learn**: show strategy stats and a suggested workflow extension.
9. Architecture walkthrough.

`scripts/seed_demo.py` replays pre-recorded observation sessions so the discovery step can also be demonstrated without re-recording (useful as a backup in the video).

6. Milestones

#	Milestone	Output	Est.
M0	Scaffolding	repo layout, FastAPI skeleton, SQLite models, Vite dashboard shell, docker-compose	0.5 d
M1	Mini-CRM	REST + HTML UI, seeded customers, API on/off toggle	0.5 d
M2	Observe	Chrome extension + OS agent → <code>/events</code> ingest, redaction, live feed	1.5 d
M3	Discover	sessionizer, abstraction, mining, clustering, scoring, data-flow links + unit tests on synthetic sequences	1.5 d
M4	Understand + Generate	Claude prompts/tool schemas, DSL, validation/retry, capability check	1 d
M5	Approval UI	candidates list, React Flow editor, dry run, approve	1 d
M6	Automate	engine, Gmail trigger, Gmail/Slack/CRM API connectors, Playwright CRM fallback, HITL pause/resume	2 d
M7	Learn	strategy stats, corrections, extension suggestions	0.5 d
M8	Hardening + docs	tests, README (install/configure/run), <code>.env.example</code> , seed script, demo recording	1 d

Critical path: M2 → M3 → M4 → M6. If time is short, cut accessibility/vision strategies to stubs with a clear interface (keep API + Playwright fully working), and keep the OS agent to window + file events.

7. Testing

- Unit: mining/clustering on synthetic event streams (known pattern injected with noise must be recovered); DSL validation; templating; condition evaluation.
- LLM: fixture-based tests with recorded Claude responses; schema-validation retry path.
- Integration: engine against Mini-CRM + mocked Gmail/Slack; Playwright fallback test against Mini-CRM UI.
- E2E smoke: `seed_demo.py` → discover → approve → run, asserted via DB state.

8. Risks & mitigations

Risk	Mitigation
OS-level capture differs per OS	Browser extension carries the demo; OS agent limited to portable signals
LLM output malformed/hallucinated steps	Strict tool schema, Pydantic validation, capability check, human approval gate
UI selectors break	Semantic selectors (role/label), LLM selector repair, API-first ladder
Gmail/Slack OAuth setup friction for judges	<code>.env.example</code> , step-by-step README, mock connectors via <code>DEMO_MODE=offline</code>
Privacy concerns	Local-only store, redaction, no raw keystrokes, pause toggle, allow-list

9. Open questions

- Target OS for the demo machine (affects which accessibility backend we fully implement).
- Real CRM (e.g. HubSpot free tier) vs. bundled Mini-CRM — plan assumes Mini-CRM for reliability, with the connector interface ready for HubSpot.
- Hackathon deadline — the milestone estimates (~9.5 dev-days) should be compressed or scoped accordingly.